LISTADO DE TABLA Y PEQUEÑA INTRODUCCIÓN A JPQL

Volvemos al proyecto del primer día y modificamos el código principal para crear cuatro elementos. Borra la base de datos que ya tenías para no tener problemas. Sustituye esto en tu main y ejecútalo:

```
try {
   em.getTransaction().begin();
   Producto p = new Producto("Teclado", 25.50);
   em.persist(p);
   p = new Producto("Ratón", 12.75);
   em.persist(p);
   p = new Producto("Fuente de alimentación", 28.24);
   em.persist(p);
   p = new Producto("Monitor 21\"", 108.32);
   em.persist(p);
   em.getTransaction().commit();
}catch(Exception e) {
   System.out.println("Error: " + e.getMessage());
   em.getTransaction().rollback();
}
```

Esto creará cuatro elementos en nuestra tabla de productos. Vamos ahora a listarlos. Primero añadimos un método toString() a nuestra clase Producto (no es necesario, pero asi queda mejor):

```
@Override
public String toString() {
      return "- Nombre: " + nombre + "\n  Precio: " + precio + "€\n";
}
```

Y el código para listar los elementos de la tabla quedaría así:

```
try {
      // Consulta JPQL Ojo: se usa el nombre de la CLASE (Producto) y no el de
      la tabla (productos)
      String jpql = "FROM Producto";
      // Ejecutamos la consulta
      List<Producto> listaProductos = em.createQuery(jpql,
      Producto.class).getResultList();
      // Recorremos la lista y mostramos los resultados
      System.out.println("--- LISTADO DE PRODUCTOS ---");
      for (Producto p : listaProductos) {
            System.out.println(p);
            }
      System.out.println("Total de productos: " + listaProductos.size());
}catch(Exception e) {
      System.out.println("Error: " + e.getMessage());
      }
```

En JPQL no hace falta poner SELECT * si vamos a leer el objeto completo. Basta con poner FROM Producto como ves en el ejemplo.
CreateQuery ejecuta la consulta y getResultList recoge el resultado y lo devuelve en formato List. El resto ya lo conoces!

Algunas variantes de la consulta anterior para ordenar los resultados o filtrar los resultados obtenidos. Fijate que trabajmos con los atributos de la clase y los objetos y no con los elementos de la base de datos

```
String jpql = "FROM Producto p WHERE p.precio < 50";

String jpql = "FROM Producto p ORDER BY p.precio DESC";
String jpql = "FROM Producto p ORDER BY p.nombre ASC, p.precio DESC";

String jpql = "FROM Producto p WHERE p.nombre LIKE 'Fuente%'";
```

También podemos parametrizar las querys de forma muy parecida a la que usábamos con PreparedStatement:

```
String jpql = "FROM Producto p WHERE p.precio > :precioMinimo";
List<Producto> listaProductos = em.createQuery(jpql, Producto.class)
      .setParameter("precioMinimo", 50.0)
      .getResultList();
```

Una consulta un poco mas compleja con dos parámetros:

```
String jpql = "FROM Producto p WHERE p.nombre LIKE :nombre AND p.precio
      <= :precioMax ORDER BY p.precio ASC";
List<Producto> listaProductos = em.createQuery(jpql, Producto.class)
      .setParameter("precioMax", 150.00)
      .setParameter("nombre", "Tecla%")
      .getResultList();
```

Como vemos, esta nueva forma de crear parámetros es mas cómoda porque les asignamos nombres diferenciadores. Aunque si lo prefieres puedes usar un método similar al que ya conoces:

```
String jpql = "FROM Producto p WHERE p.nombre LIKE ?1 AND p.precio <= ?2
      ORDER BY p.precio ASC";
List<Producto> listaProductos = em.createQuery(jpql, Producto.class)
      .setParameter(2, 150.00)
      .setParameter(1, "Tecla%")
      .getResultList();
```

Este método de encadenar resultados sin necesitar variables intermedias se conoce como Fluent API. Lo hemos usado alguna vez, pero no de forma tan compleja como aquí. Conviene que te acostumbres porque es la forma mas profesional de trabajar ¿para que usar variables intermedias que no necesitas? En lugar de clarificar hacen el código mas complejo.
Para que lo veas, vamos a desarrollar esta última instrucción de forma convencional:

```
String jpql = "FROM Producto p WHERE p.nombre LIKE ?1 AND p.precio <= ?2
      ORDER BY p.precio ASC";
TypedQuery<Producto> query1 = em.createQuery(jpql, Producto.class);
TypedQuery<Producto> query2 = query1.setParameter(1, "Tecla%");
TypedQuery<Producto> query3 = query2.setParameter(2, 150.00);
List<Producto> listaProductos = query3.getResultList();
```

Si sólo necestamos recoger el nombre y no toda la información del producto, este sería nuestro código:

```
String jpql = "SELECT p.nombre FROM Producto p";
List<String> listaNombres = em.createQuery(jpql, String.class).getResultList();
System.out.println("--- LISTADO DE NOMBRES DE PRODUCTOS ---");
for (String nombre : listaNombres) {
      System.out.println(nombre);
      }
System.out.println("Total de productos: " + listaNombres.size());
```

Un último ejemplo ¿Y si necesitamos varios elementos pero no la clase completa? En el siguiente recuperamos el nombre y el precio

```
String jpql = "SELECT p.nombre, p.precio FROM Producto p";
List<Object[]>          listaResultados          =          em.createQuery(jpql,
Object[].class).getResultList();
System.out.println("--- LISTADO DE VARIOS ELEMENTOS DE LOS PRODUCTOS ---");
for (Object[] fila : listaResultados) {
      String nombre = (String) fila[0];   // Posición 0: p.nombre
      Double precio = (Double) fila[1];   // Posición 1: p.precio
      System.out.println("Producto: " + nombre + " | Precio: " + precio + "€");
      }
System.out.println("Total de resultados: " + listaResultados.size());
```

NOTA: En este caso nuestros objetos tienen solo un elemento mas aparte de estos dos (el id) así que este planteamiento es absurdo pero imagina que trabajas con una tabla con decenas de columnas y que solo necesitas la información de dos de ellas. En ese caso este planteamiento sería el mas correcto

GENERACIÓN AUTOMÁTICA DE CLASES

Es posible que hibernate genere automáticamente las clases necesarias para operar con una determinada base de datos de forma muy sencilla. Eclipse ha dejado de incluir un plugin que lo hacía, pero veremos una forma sencilla de crear un proyecto que las genere usando el plugin hbm2java que aunque no viene con Eclipse se puede invocar a través de un proyecto Maven. Vamos a usar como ejemplo la base de datos pokemondb que ya conocemos y que debería de estar instalada en nuestro servidor.

Empezamos por crear un proyecto Maven igual que el anterior. En este caso he elegido los siguientes nombres:

group id: com.dam2.goya
artifact id: adat-jpa2

En el fichero pom.xml copiamos esta vez lo siguiente:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
	xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
	xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
	https://maven.apache.org/xsd/maven-4.0.0.xsd">
	<modelVersion>4.0.0</modelVersion>
	<groupId>com.dam2.goya</groupId>
	<artifactId>adat-jpa2</artifactId>
	<version>0.0.1-SNAPSHOT</version>

	<properties>
		<maven.compiler.release>17</maven.compiler.release>
		<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
		<hibernate.version>7.4.14.Final</hibernate.version>
	</properties>

	<dependencies>
		<dependency>
			<groupId>org.slf4j</groupId>
			<artifactId>slf4j-nop</artifactId>
			<version>2.0.16</version>
		</dependency>
		<dependency>
			<groupId>org.hibernate.orm</groupId>
			<artifactId>hibernate-core</artifactId>
			<version>${hibernate.version}</version>
		</dependency>
		<dependency>
			<groupId>com.mysql</groupId>
			<artifactId>mysql-connector-j</artifactId>
			<version>9.1.0</version>
		</dependency>
	</dependencies>
</project>
```

Recuerda cambiar la parte correspondiente al group id y artifact id en tus propios proyectos. También lo relativo al driver de mysql por mariadb u otro gestor si es el caso.

A continuación creamos un archivo dentro de la carpeta src/main/resources que se llame hibernate.properties. El contenido de este archivo será el siguiente:

```
hibernate.connection.driver_class=com.mysql.cj.jdbc.Driver
hibernate.connection.url=jdbc:mysql://localhost:3306/pokemodb
hibernate.connection.username=profe
hibernate.connection.password=abc123
hibernate.default_catalog=pokemondb
```

Sustituye las dos primeras líneas por las siguientes si usas mariadb:

```
hibernate.connection.driver_class=org.mariadb.jdbc.Driver
hibernate.connection.url=jdbc:mariadb://localhost:3306/pokemondb
```

Y no olvides cambiar usuario, password y base de datos (en los dos lugares donde aparece) si quieres usar este método con otra diferente.

Ahora pulsa sobre el proyecto con el botón derecho. Elige Run As – Maven build y en la ventana que aparece haz los siguientes ajustes:

En la pantalla de Main, en el text edit de Goals escribe lo siguiente:

```
org.hibernate.orm:hibernate-maven-plugin:7.4.14.Final:hbm2java
```

Pulsa Apply y luego Run. Las clases generadas se crearan en la carpeta target\generated-sources

NOTA: Algunas versiones de Java pueden dar problemas. La versión minima de Java recomedadaa tiene que ser la 17 o superior. Pulsa en la pestaña JRE y asegurate de que es así. Si no tienes elegida una opción por defecto que cumpla esto pulsa en Alternate JRE y busca si la tienes instalada en tu sistema. En caso contrario tendrás que solucionarlo instaladola antes de nada.

En algunos casos puedes encontrarte que se generan mas clases que tablas. Esto es debido fundamentalmente a que a veces son necesarias entidades adicionales para las relaciones ManyToMany con atributos adicionales, para las claves primarias compuestas, etc.

Verás también que el plugin genera alguna anotación que está desaconsejada (deprecated). Las reconocerás porque aparecen tachadas en el código:

```
@GenericGenerator(name="MoIdGenerator", strategy="foreign")@Id
@GeneratedValue(generator="MoIdGenerator")
```

Ya veremos mas adelante como corregir esto. Échale un vistazo al código generado y, aunque no lo entiendas completamente, plantéate una cuestión: cuando tengas que diseñar una aplicación que usa una base de datos compleja (como esta) que te resulta mas sencillo:

A) diseñar la base de datos y que se te generen las clases de forma automática como hemos hecho ahora

B) crear las clases a mano y que estas generen las tablas de forma automática como hicimos ayer

La A) ¿verdad?

ANOTACIONES Y SU USO

Las principales anotaciones usadas en Hibernate son las siguientes. Algunas ya las conoces y otras aparecen en el código que has generado antes:

@Entity	Indica que la clase es una entidad de persistencia y se mapeará a la base de datos
@Table	Especifica detalles de la tabla física
@Id	Declara el atributo que será clave primarias
@GeneratedValue	Define la estrategia de generación automática para la clave primaria
@Column	Permite personalizar el mapeo del atributo con la columna correspondiente de la tabla
@OneToOne	Relación asociativa de Uno a Uno
@ManyToOne	Relación asociativa de Muchos a Uno
@OneToMany	Relación asociativa de Uno a Muchos
@ManyToMany	Relación asociatia de Muchos a Muchos
@JoinColumn	Especifica la columna de unión en una foreign key
@JoinTable	Configura los detalles de la tabla intermedia en relaciones Muchos a Muchos
@Transient	Indica que un atributo de la clase no se guardará en la base de datos